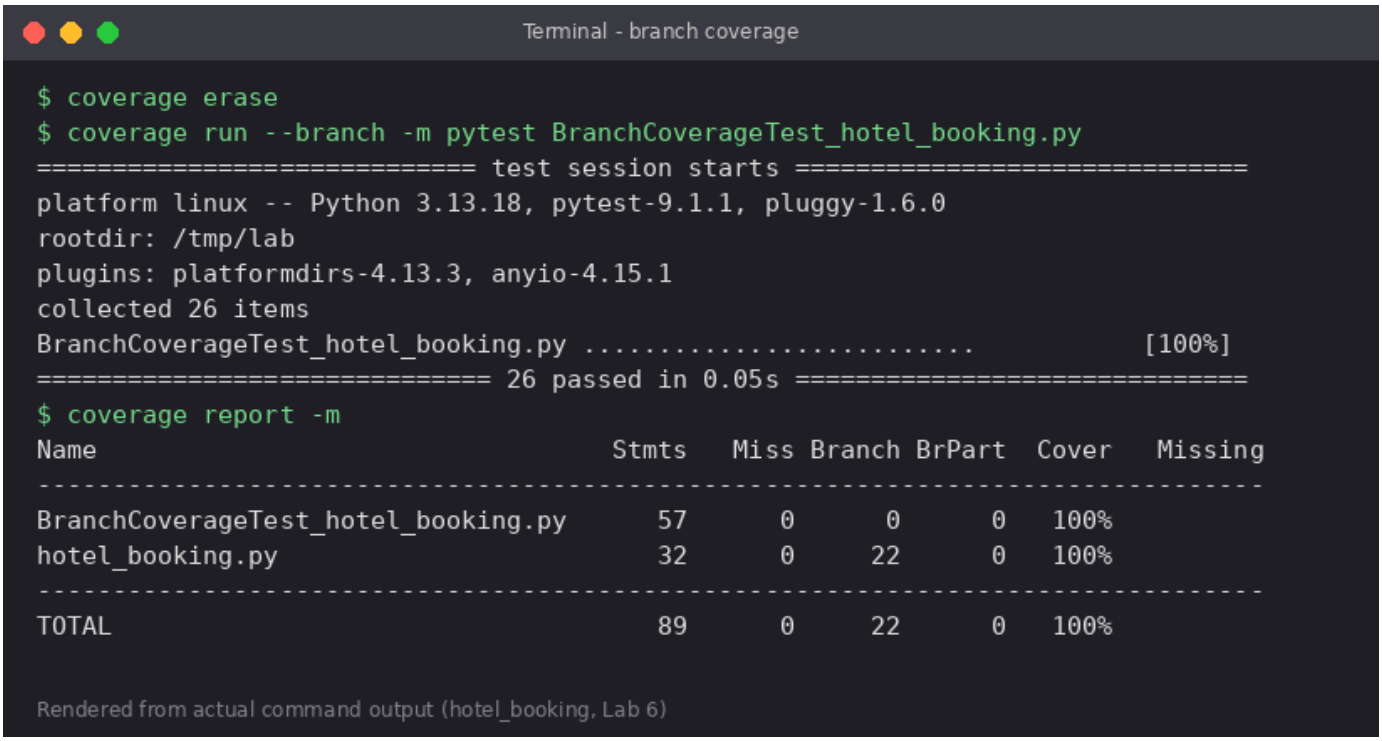
# Branch Coverage - 100%

Lab 6: Code and Branch Coverage - hotel_booking (6_L7_22_hotel_booking)

### Commands run:

```
coverage run --branch -m pytest BranchCoverageTest_hotel_booking.py
coverage report -m
```



## Test file: BranchCoverageTest_hotel_booking.py

```
"""Lab 6 - Branch coverage tests for hotel_booking.py"""
import pytest
from hotel_booking import booking_cost, cancellation_refund, room_upgrade


# ---------- booking_cost ----------
def test_standard_two_nights():
    assert booking_cost("standard", 2, 2, False) == 4000


def test_unknown_room_raises():
    with pytest.raises(ValueError, match="unknown room"):
        booking_cost("penthouse", 2, 2, False)


def test_zero_nights_raises():
    with pytest.raises(ValueError, match="invalid nights"):
        booking_cost("standard", 0, 2, False)


def test_negative_nights_raises():
    with pytest.raises(ValueError, match="invalid nights"):
        booking_cost("deluxe", -3, 2, False)


def test_one_night_boundary_is_valid():
    assert booking_cost("deluxe", 1, 1, False) == 3500


def test_peak_season_surcharge():
    # 2000 + 500 = 2500 per night x 2
    assert booking_cost("standard", 2, 2, True) == 5000


def test_off_peak_no_surcharge():
    assert booking_cost("suite", 1, 2, False) == 6000


def test_extra_guests_charge():
    # 4000 + (3-2)*500*2 = 5000
    assert booking_cost("standard", 2, 3, False) == 5000


def test_exactly_two_guests_no_extra():
    assert booking_cost("standard", 3, 2, False) == 6000


def test_six_nights_no_long_stay_discount():
    assert booking_cost("standard", 6, 2, False) == 12000


def test_seven_nights_discount_boundary():
    # 14000 - 1400 = 12600
    assert booking_cost("standard", 7, 2, False) == 12600


def test_all_conditions_combined():
    # rate 7500 x 7 = 52500 + 7000 = 59500 - 5950 = 53550
    assert booking_cost("suite", 7, 4, True) == 53550


# ---------- cancellation_refund ----------
def test_full_refund():
    assert cancellation_refund(1000, 10, False) == 1000


def test_full_refund_boundary_7_days():
    assert cancellation_refund(1000, 7, False) == 1000


def test_half_refund_non_member():
    assert cancellation_refund(1000, 3, False) == 500


def test_half_refund_boundary_2_days():
    assert cancellation_refund(1000, 2, False) == 500


def test_half_refund_member_bonus():
    assert cancellation_refund(1000, 5, True) == 750
```

```
def test_no_refund_non_member():
    assert cancellation_refund(1000, 1, False) == 0


def test_no_refund_member_bonus_only():
    assert cancellation_refund(1000, 0, True) == 250


def test_full_refund_member_unchanged():
    assert cancellation_refund(1000, 10, True) == 1000


# ---------- room_upgrade ----------
def test_no_rooms_free_returns_same_room():
    assert room_upgrade("standard", True, 0) == "standard"


def test_standard_member_upgrades_to_deluxe():
    assert room_upgrade("standard", True, 2) == "deluxe"


def test_standard_non_member_no_upgrade():
    assert room_upgrade("standard", False, 2) == "standard"


def test_deluxe_member_upgrades_to_suite():
    assert room_upgrade("deluxe", True, 2) == "suite"


def test_deluxe_non_member_no_upgrade():
    assert room_upgrade("deluxe", False, 2) == "deluxe"


def test_suite_member_stays_suite():
    assert room_upgrade("suite", True, 2) == "suite"
```